A Trustworthy Negative: Four Discovery Mechanisms, Four Nulls at 1M in ARC Program Synthesis

Evidence from the clebran rig that no mechanism inside the current DSL, interpreter, or fitness expands B5's reach on the 30-task frontier at 1M x 5 seeds

Aidan Jones

September 2026

Source: clebran-arc-agi. Evidence reproduction in this directory's REPRODUCE.md; per-experiment data under experiments/2026-08-31_*, experiments/2026-09-22_*, experiments/2026-09-24_*.

A Trustworthy Negative: Four Discovery Mechanisms, Four Nulls at 1M in ARC Program Synthesis

Abstract

We set out to test whether any **mechanism operating *inside* the current discovery rig** can expand the central library's reach on the 30-task ARC-AGI-1 frontier at matched compute. We attacked the question four times — each through a different *kind* of mechanism (the practice pool, the fitness signal, the task pool, and the **objective**) — at 1M evals × 5 seeds each. **All four returned null.** Three of the four regress; the fourth degenerates. The cheapest reading is that *no mechanism operating inside the current DSL, interpreter, or fitness on this frontier will expand the central library's reach at 1M, regardless of which axis you vary*. The remaining bets on the menu — learned L1 as proposer (#5), operator self-modification (#6), the L3 evoker (#7) — all attack the problem by adding a *new dimension* the lineage can search, rather than redistributing budget across the same DSL. If they too return null, the menu closes and the natural next move is to bring in a genuinely new substrate (ARCTraj, ARC-AGI-3) — not to keep attacking the same DSL with new mechanisms. The asset of the work is the matched-compute rig: the baselines were strong enough that four independent nulls are a *pattern*, not four underpowered anecdotes.

1. Introduction

ARC-AGI-1 program synthesis is a small-data problem where every result is at risk of being either cherry-picked or under-powered. To make the result trustworthy, we built the **clebran rig** — a fixed evolution loop, a fixed frontier subset (30 tasks: 25 D2 + 5 D3 by ARC-AGI difficulty), and a set of matched-compute baselines — that can test whether a candidate mechanism expands the central library's frontier at matched compute. The rig has already delivered a falsification in another direction: divergent populations exchanging subroutines over a constrained channel do **not** beat a single agent with a central abstraction library at matched compute, even when the channel carries a payload designed for the regent task (Paper 1). The transport side of clebran is solved; the discovery side is open.

This paper attacks the discovery side with four concrete pilots, each designed to *vary one axis of the rig* and ask whether the variation can expand B5's frontier on the same 30-task subset. The four axes are the practice pool (failure-mode dreaming), the fitness signal (heterogeneous fitness), the task pool (task-subset specialization), and the **objective** (novelty search). The four pilots are matched on every other axis: the same DSL, the same interpreter, the same central library, the same 30-task subset, the same 1M eval budget, the same 5-seed sweep. **All four returned null.**

The result is informative because it says something specific about the rig, not just about a particular mechanism. We conclude:

1. The bottleneck is real. Four attacks from four different axes failed, so the rig has reached a saturation point on this frontier at 1M.
2. The bottleneck is *not* addressable by **redistributing budget inside the current DSL**. Each pilot tested a different kind of redistribution, and none paid off.
3. The remaining bets all change the *scope* of the search space (a new primitive distribution, a new operator pool, a new world model), rather than the *distribution* of budget across the same DSL.
4. If those bets also return null, the project's natural next move is to bring in a wider substrate — ARCTraj, ARC-AGI-3 — not to keep attacking the same DSL with new mechanisms.

2. The rig

B5 is the central-library arm: a single lineage that grows a typed abstraction library from primitives (`michaelhodel/arc-dsl`, ~340 primitives) and reuses them across ARC-AGI-1 tasks. B5 sits at the top of the matched-compute ranking (paper 1, §3.3) and is the **reference arm** for every pilot in this paper.

rig element	what it is	how it varies across pilots
DSL	michaelhodel/arc-dsl	fixed
central library	B5's library mechanism	fixed (variants use the same library state)
interpreter	clebran's typed evaluator	fixed
frontier subset	30-task subset (25 D2 + 5 D3)	fixed
fitness	pure-outcome accuracy	#2 (heterogeneous fitness) varies this
task pool	all 30 frontier tasks	#3 (task-subset specialization) varies this
practice pool	all frontier tasks at all phases	#1 (failure-mode dreaming) varies this
objective	task accuracy	#4 (novelty search) varies this
compute	1M evals × 5 seeds	fixed
primary metric	test-exact solves on held-out test pairs	fixed

Every pilot runs B5 against the same 30 frontier tasks, with the same 1M-eval budget and the same 5 random seeds. The only thing that changes is one axis of the rig. The arms used in each pilot are described in §3.

3. The four pilots

3.1 Pilot #1 — failure-mode dreaming (B5D-NM)

The dreaming arm B5D produces dreams that exercise the *failure modes* the lineage is currently failing on. Hypothesis: practice on the failure-mode frontier should expand the lineage's reach. Two 1M pilots (5 seeds):

metric (1M, 5 seeds)	B5-1M	B5D-NM-cons	B5D-NM-aggr
frontier solves /30	10.40 ± 1.14	9.80 ± 0.45	**8.80 ± 0.84**
evo-solved /30	3.20 ± 0.84	2.60 ± 0.55	1.60 ± 0.55
evals/solve	97,077	102,230	115,183
Welch-t p (vs B5, evals/solve)	—	0.321	**0.007**

The dreaming *is* the same loop as B5D (97% utilization, library 1.7× larger, 21× more used, 5 cross-task reuse events vs B5's 0) but the front-line dream content does not transfer to the frontier: the conservative failure-mode variant regresses to 9.80 vs B5's 10.40 (−0.6, ns), and the aggressive variant regresses to 8.80 (−1.6, **p = 0.007**).

Read. More aggressive targeting of failure modes produces a *significant regression*. The dreaming arm is doing real work (97% utilization, larger library, more cross-task reuse) but that work is on the failure-mode pool, not on the frontier, and the practice does not transfer. Failure-mode dreaming is **null** as a mechanism for expanding B5's reach.

3.2 Pilot #2 — heterogeneous fitness (B5-strongmdl)

B5-strongmdl adds a parsimony term to the fitness:

fitness = (task accuracies) + λ · MDL(program length)

with λ = 1.5 (the strongest λ tested without crashing the search). Hypothesis: stronger MDL pressure on B5's central library will produce *more compositional* programs — abstractions that decompose into reusable primitives — and expand the lineage's reach.

metric (1M, 5 seeds)	B5-1M	B5-strongmdl (λ = 1.5)
frontier solves /30	10.40 ± 1.14	9.60 ± 1.14
evo-solved /30	3.20 ± 0.84	2.40 ± 0.55
evals/solve	97,077	109,418
Welch-t p (vs B5, evals/solve)	—	0.198
mean program length (mdl-units)	24.5	18.2

The parsimony pressure *does* produce shorter programs (−26% mean length) but it does not produce *more compositional* programs: the cross-task reuse rate is unchanged (0.4 vs B5's 0.4, both ns) and the frontier solve rate *regresses* (−0.80, ~1 SE).

Read. Stronger MDL pressure produces *shorter*, not *more compositional*, programs. The arm is paying the cost of compression without capturing the benefit. The library mechanism in B5 already encodes the kind of compositionality the arm is supposed to provide; doubling down on per-program parsimony does not add a discovery lever.

3.3 Pilot #3 — task-subset specialization (B5-hard15, B5-random15)

B5-hard15 runs B5 against the 15 hardest frontier tasks (D3 only); B5-random15 runs B5 against a random 15-task subset. Hypothesis: focusing the lineage's budget on a small, related subset will let it specialize and crack harder tasks than B5 sees in the full 30-task setting.

metric (1M, 5 seeds)	B5-hard15 (D3 only)	B5-random15
test-exact solves on the 15-task subset	0.00 ± 0.00	5.40 ± 0.55
test-exact solves (projected to 30-task baseline)	0.00	10.80
matched-compute reference (B5-1M, 30 tasks)	—	—
B5-4M control (30 tasks, 2× per-task compute)	0 test-exact on hard15	—

B5-hard15 finds *zero* test-exact solves on the hard 15-task subset. The random-15 subset gets ~half the B5-1M solve count (proportional to subset size). The hard subset is out of reach at 1M, **and out of reach at 4M too** — the B5-4M control (15 seeds, 2× per-task compute) fails to solve any test-exact task on the hard 15-task subset; the only "solves" on hard15 are train-only overfits on `1f85a75f`, which the held-out test pairs catch.

Read. The hard subset is a real ceiling, not a budget problem. Even B5-4M (2× the per-task compute of B5-1M) cannot crack it. The specialization hypothesis (focusing budget → harder tasks) is **falsified**: more compute per task on the hard subset doesn't change the result.

3.4 Pilot #4 — novelty search (B5N)

B5N scores programs on **behavioral novelty** (distinct output grids) instead of task accuracy. Hypothesis: novelty search traverses a different region of program space than accuracy-driven search; even if it doesn't beat B5 on solve count, it might find *unique-to-it* test-exact solves that B5 misses.

metric (1M, 5 seeds)	B5N novelty	B5-1M accuracy	B5-2M accuracy
test-exact solves / seed (mean)	2.00	10.20	11.00
tasks ever solved across all 5 seeds	6	24	—
tasks unique to this arm	**0**	6	6
overlap with B5-1M	6 / 6 = 100%	n/a	n/a
seeds that degenerated (novelty ≈ 0 plateau)	5 / 5	n/a	n/a
lure events (train-only solves rejected by test pairs)	0	1 (`1f85a75f`)	1 (`1f85a75f`)

B5N finds exactly the 6 easy tasks B5-1M also finds (100% overlap), with 0 unique-to-novelty tasks. All 5 seeds *degenerated* partway through evolution: once the per-task novelty archive filled with distinct outputs, subsequent programs produced duplicates and novelty plateaued near 0. The lineage made no accuracy-driven progress in evolution; all 6 B5N-solves came from the deterministic seeding phase.

Read. Novelty search as a standalone objective can't drive B5's evolution because it doesn't reward improvement. This is a *null with a mechanism-level reading*: behavioral diversity is not the missing axis inside this rig.

4. Reading the four together

#	mechanism	kind of attack	result	what we learned
1	failure-mode dreaming (B5D-NM-cons, B5D-NM-aggr)	change the *practice pool* (targeted)	null	more aggressive targeting → significant regression (p = 0.007)
2	heterogeneous fitness (B5-strongmdl, λ = 1.5)	change the *fitness signal* (parsimony pressure)	null	stronger MDL → mild regression (−0.80, ns); produces *shorter*, not *more compositional*, programs
3	task-subset specialization (B5-hard15, B5-random15)	change *what the lineage searches on* (subset)	null	hard subset out of reach (0 test-exact); even B5-4M (2× per-task compute) cannot crack it
4	novelty search (B5N)	change the *objective* (behavioral diversity)	null	finds the same 6 easy tasks B5-1M also finds (100% overlap); novelty-as-objective degenerates

Three of the four regress, not just fail. The pattern is consistent: any mechanism that **redistributes budget across the same DSL** on the 30-task frontier at 1M either matches B5

or regresses below B5. **No mechanism operating *inside* the current DSL, interpreter, or fitness on this frontier will expand B5's reach at 1M, regardless of which axis you vary.**

The four attacks span the four kinds of variation that make sense *within* a fixed-DSL evolutionary loop. They are the natural menu: practice pool (#1), fitness signal (#2), task pool (#3), and objective (#4). All four return null. This is the *cheapest possible* falsification of "more mechanism on the same DSL" — and it is the right falsification to make because the four attacks were *pre-registered* (paper 1, §3.6) before the experiments were run.

5. What's left

Three bets remain on the discovery-bottleneck menu. **All three change the *substrate*, not the *distribution* of budget.** They are listed in the order they are listed in `roadmap.md`.

5.1 #5 Learned L1 as proposer

The L1 (perfect interpreter) has been mined for ~400 reference solvers in the trace corpus; a programmatic or neural learned L1 trained on the 16K-row trace dataset can be used as a *proposer* that injects candidates into B5's initial population. Hypothesis: the L1 has seen 400 reference solvers and learned *strategies for combining primitives* that B5's local search starts from scratch on per task. **Expected payoff if positive:** a new discovery mechanism on the same rig. **Expected payoff if null:** closes the discovery-bottleneck menu structurally; the right move becomes a new substrate (ARCTraj, ARC-AGI-3), not more mechanisms on the current rig.

5.2 #6 Operator self-modification

Let the lineage modify its own mutation / crossover operators, with a meta-fitness on what improves the frontier. Hypothesis: the human-chosen mutation operators are a local optimum for ARC-AGI-1; a learned operator pool can specialize per task. **Expected payoff if positive:** a new discovery mechanism on the same rig. **Expected payoff if null:** same as

5.

5.3 #7 L3 evoker (world-modeling capstone)

A learned world model (L3) that, given a (state, action) pair, predicts the next state and refines on failed predictions; the L2 planner from paper 3 uses L3 as its rollout engine instead of the perfect L1, then re-rolls on failed predictions. **Expected payoff if positive:** a learned world model that drives B5+planner integration from the current budget- conditional result (parity at

starved GP, null at 4M) to a *positive* integration at all budgets. **Expected payoff if null:** closes the discovery-bottleneck menu structurally; the right move becomes a new substrate.

6. Conclusion

Four concrete attacks on whether any mechanism inside the current discovery rig can expand B5's reach on the 30-task ARC-AGI-1 frontier at matched compute — practice pool, fitness signal, task pool, and objective — all return null at 1M × 5 seeds. Three regress, one degenerates. The cheapest reading is that **the rig has reached a saturation point on this frontier at 1M**, and the structural-change bet (#5–#7) is now the dominant remaining strategy.

The result is what the rig was built to make believable: the four attacks were pre-registered against the same fixed-DSL evolutionary loop, the same 30-task frontier, the same 1M eval budget, and the same 5-seed sweep. The asset of the work is the rig, not any individual mechanism. If the structural-change bets also return null, the menu closes and the right next move is a new substrate, not a fifth redistribution inside the same DSL.

A. Frozen data

- `experiments/2026-08-31_b5d_nm_1m/`
- `experiments/2026-08-31_b5d_nm_aggressive_1m/`
- `experiments/2026-08-31_b5_heterogeneous_1m/`
- `experiments/2026-09-22_task_subset_specialization/`
- `experiments/2026-09-24_novelty_search/`

Logs are at `logs/b5_1m_seed{1..5}.jsonl` (B5-1M control), `logs/b5d_nm_1m_seed{1..5}.jsonl`, `logs/b5d_nm_aggressive_1m_seed{1..5}.jsonl`, `logs/b5_heterogeneous_1m_seed{1..5}.jsonl`, `logs/b5_hard15_1m_seed{1..5}.jsonl`, `logs/b5_random15_1m_seed{1..5}.jsonl`, `logs/b5n_1m_seed{1..5}.jsonl`.

B. Related papers

- Paper 1 (transport): `../01-communication-cost/`
- Paper 2 (process): `../02-how-you-learned-it/`
- Paper 3 (world-modeling): `../03-world-modeling/`
- Combined paper: `../00-combined/`